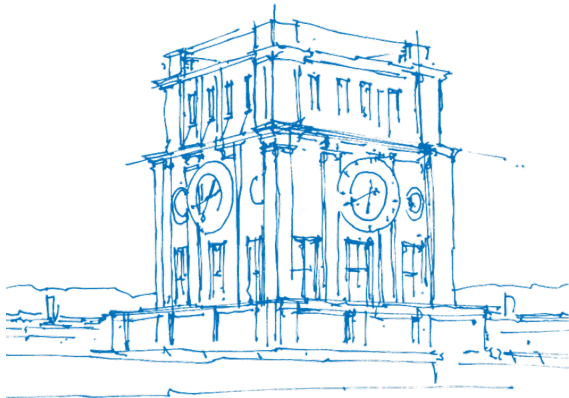


# GRA-Projekt A207

## Bildverarbeitung

**Clemens März, Lucas Michel, Justus Pansch**

23. August 2024



*TUM Uhrenturm*

- 1** Problemstellung
- 2 Lösungsansatz und Designentscheidungen
- 3 Optimierungen
- 4 Korrektheit und Genauigkeit
- 5 Performanzanalyse

# Problemstellung

- Konvertierung der Farbwerte in Graustufenwerte durch gewichteten Durchschnitt

$$q = \frac{a * r, b * g, c * b}{a + b + c}$$

- Helligkeitsanpassung

$$q' = clamp_0^{255}(q + brightness)$$

- Berechnung des Durchschnitts sowie der Standardabweichung

$$\mu = \frac{1}{|Q|} \sum_{q \in Q} q' \qquad \sigma = \sqrt{\frac{1}{|D|} \sum_{q \in Q} (q' - \mu)^2}$$

# Problemstellung

- Kontrastanpassung

$$q'' = clamp_0^{255} \left( \frac{k}{\sigma} * q' + \left(1 - \frac{k}{\sigma}\right) * \mu \right)$$

- Methode zum Wurzelziehen mit Grundoperationen
- Bestimmung von sinnvollen Standardwerten für den gewichteten Durchschnitt

# Outline

- 1 Problemstellung
- 2 Lösungsansatz und Designentscheidungen
  - Rahmenprogramm
  - Graustufenkonvertierung/Helligkeitsanpassung
  - Kontrastanpassung
- 3 Optimierungen
- 4 Korrektheit und Genauigkeit
- 5 Performanzanalyse

# Format der Eingabe Datei

## ■ Portable Pixel Map (ppm)<sup>1</sup>

### ■ Aufbau

- ☐ Magic Number: P6
- ☐ Bildgröße: Angegeben durch zwei Zahlen, Breite und Höhe in Pixeln
- ☐ Maximaler Farbwert: Definiert die Farbtiefe
- ☐ Pixel Daten: Binärdaten, jedes Pixel wird durch drei Bytes dargestellt (Rot, Grün, Blau)

### ■ Besonderheiten des Dateiformates

- ☐ Mehrere Bilder in einer Datei möglich
- ☐ Kommentare im Header sind möglich
- ☐ Maximaler Farbwert kann kleiner als 255 sein

---

<sup>1</sup><https://netpbm.sourceforge.net/doc/ppm.html>

# Format der Ausgabe Datei

## ■ Portable Gray Map (pgm)<sup>2</sup>

### ■ Aufbau

- ☐ Magic Number: P5
- ☐ Bildgröße: Angegeben durch zwei Zahlen, Breite und Höhe in Pixeln
- ☐ Maximaler Farbwert: definiert Graustufen Tiefe
- ☐ Pixel Daten: Binärdaten, jedes Pixel wird durch ein Byte dargestellt

### ■ Vorteile

- ☐ Kompakter als PPM da nur ein Byte pro Pixel benötigt wird

---

<sup>2</sup><https://netpbm.sourceforge.net/doc/pgm.html>

## Standardwerte für die Gewichte $a, b$ und $c$

- Gleiche Gewichtung aller Farbkomponenten ( $a = b = c$ )
- Gewichtung nach Farbwahrnehmung<sup>3</sup>: Das Menschliche Auge nimmt Grün stärker als Rot und Rot stärker als Blau wahr
  - Resultierende Gewichtung

$$D = 0,299 * R + 0,587 * G + 0,114 * B$$

- Zusammensetzung der Netzhaut, insbesondere der Empfindlichkeit der Zäpfchen

---

<sup>3</sup>[https://www.itu.int/dms\\_pubrec/itu-r/rec/bt/R-REC-BT.601-7-201103-I!!PDF-E.pdf](https://www.itu.int/dms_pubrec/itu-r/rec/bt/R-REC-BT.601-7-201103-I!!PDF-E.pdf)



## Spezialfall: $a = b = c = 0$

$$D = \frac{a * R + b * G + c * B}{a + b + c}$$

- Division durch null  $\rightarrow$  mathematisch nicht definiert
- Mögliche Anwendung: Alle Pixel mittels `--brightness` auf einen bestimmten Wert setzen

## Spezialfall: Negative Gewichte

### ■ Möglichkeiten:

- ☐ Betrag
- ☐ Invalide Eingabe

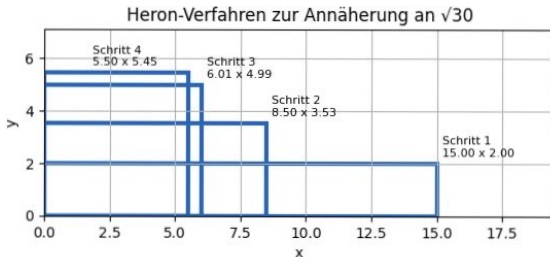
$$D = \frac{a * R + b * G + c * B}{a + b + c}$$

- Negative Parameter führen zum Fall  $a + b + c = 0$  mit  $a, b, c \neq 0 \rightarrow$  undefiniert
- Es können sowohl negative als auch zu große Werte berechnet werden  $\rightarrow$  kein gewichteter Durchschnitt mehr
- Keine erweiterte Funktionalität

# Wurzel-Berechnung mit Grundoperationen

## Heron Verfahren

- Wir verwenden das Heron-Verfahren<sup>4</sup> zur Annäherung der Quadratwurzel
- Geometrisch betrachtet, versucht man die Seiten eines Rechtecks iterativ anzugleichen  
→ wird zum Quadrat → Seitenlänge ist die Wurzel des Flächeninhalts
- $x' = \frac{1}{2} * (x + \frac{A}{x})$



<sup>4</sup><https://de.wikipedia.org/wiki/Heron-Verfahren>

## Spezialfall: Standardabweichung $\sigma = 0$

$$\frac{k}{\sigma} * q + \left(1 - \frac{k}{\sigma}\right) * \mu$$

- Mathematisch für  $\sigma = 0$  undefiniert
- In der Praxis für  $\sigma \approx 0$  nicht mehr berechenbar
- Standardabweichung  $\sigma = 0 \rightarrow$  Alle haben Pixel die gleiche Farbe, also den Durchschnitt  
 $\forall q \in Q : q = \mu$
- Kontrastanpassung basiert auf Abweichung vom Mittelwert  $\rightarrow$  Kontrast kann nicht angepasst werden
- Jedes Pixel behält seinen Wert oder wird mit dem Durchschnitt überschrieben

# Outline

- 1 Problemstellung
- 2 Lösungsansatz und Designentscheidungen
- 3 Optimierungen**
  - SIMD
  - Mathematische Umformung der Standardabweichung
  - Vorberechnungen
  - Festkommarechnung
- 4 Korrektheit und Genauigkeit
- 5 Performanzanalyse

# Anwendbarkeit von SIMD

- Datenparallelität: Pixel können unabhängig voneinander verarbeitet werden
- Gemeinsame Operationen: Die Operationen sind für alle Pixel gleich
- Kontinuierliche Speicherzugriffe: Die Daten sind hintereinander gespeichert und können effizient geladen und gespeichert werden
  - Durch Parallelisierung mit SIMD steigt die Verarbeitungsgeschwindigkeit

## Standardabweichung $\sigma$ - Umformung

- Umstellung der Formel für  $\sigma^2$  führt zu Form bei der die Summe nur über Integer gebildet werden muss

$$\begin{aligned}\sigma^2 &= \frac{1}{|D|} \sum_{q \in Q} (q - \mu)^2 = \frac{1}{|D|} \sum_{q \in Q} (q^2 - 2q\mu + \mu^2) \\ &= \frac{1}{|D|} \left( \sum_{q \in Q} q^2 - \sum_{q \in Q} 2q\mu + \sum_{q \in Q} \mu^2 \right) = \frac{1}{|D|} \left( \sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)\end{aligned}$$

- $Q$  entspricht der Menge der Pixelwerte

## Standardabweichung $\sigma$ - Vorteile

$$\sigma^2 = \frac{1}{|D|} \left( \sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)$$

- Pixelwerte  $q \in Q$  haben Typ `uint8_t`
- Vorteile:
  - SIMD: höhere Parallelisierbarkeit da mehr Werte in einen Vektor passen
  - Arithmetische Operationen innerhalb der Schleife sind performanter



## Quadrate in der Standardabweichung

$$\sigma^2 = \frac{1}{|D|} \left( \sum_{q \in Q} q^2 - 2\mu \sum_{q \in Q} q + |D|\mu^2 \right)$$

- Pixelwerte  $q \in Q$  haben Typ `uint8_t`
- Abspeichern der Quadratzahlen in einem Array
- Array-Zugriff potentiell schneller als Quadrieren
- Operationen innerhalb der Summation performanter

# Festkommazahlen

- Festkommaarithmetik durch Integer Datentypen
- Konvertierung zwischen Festkomma- und Fließkommazahlen
  - ☐ Festlegen der Nachkommastellen  $n$
  - ☐ Multiplizieren der Fließkommazahlen mit  $2^n$  und speichern in Integern
  - ☐ Rechnen
  - ☐ Rückkonvertierung auf Ganzzahlen (durch Rechts-Shift um  $n$  sowie evtl. Rundung)
- Konvertierung von Fließkomma- zu Festkommazahlen nur außerhalb von Schleifen
- Integerarithmetik in Schleifen

# Kontrastanpassung - Festkomma SIMD

- Parallelisierte Version der Kontrastanpassung rechnet mit Fließkommazahlen(  $k$ ,  $\sigma$  und  $\mu$  sind Fließkommazahlen)

- ☐ Langsam da weniger Parallelisierung möglich ist
- ☐ Langsam durch Fließkommaarithmetik

- Mit 8.8 Festkommazahlen

- ☐ Integer Arithmetik
- ☐ 16 statt 32 Bit pro Wert

→ höherer Speedup

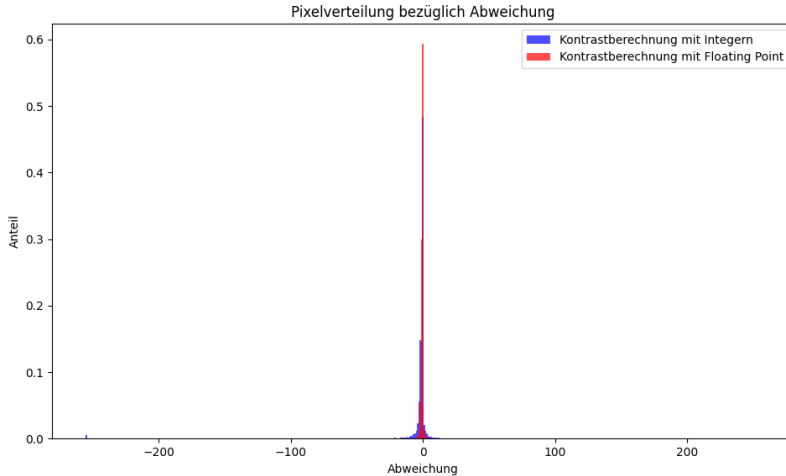
$$\frac{k}{\sigma} * q + (1 - \frac{k}{\sigma}) * \mu = \frac{k}{\sigma} * q + \mu - \frac{k}{\sigma} * \mu = \frac{k}{\sigma} * q - \frac{k}{\sigma} * \mu + \mu = \frac{k}{\sigma} * (q - \mu + \frac{\sigma}{k} * \mu)$$

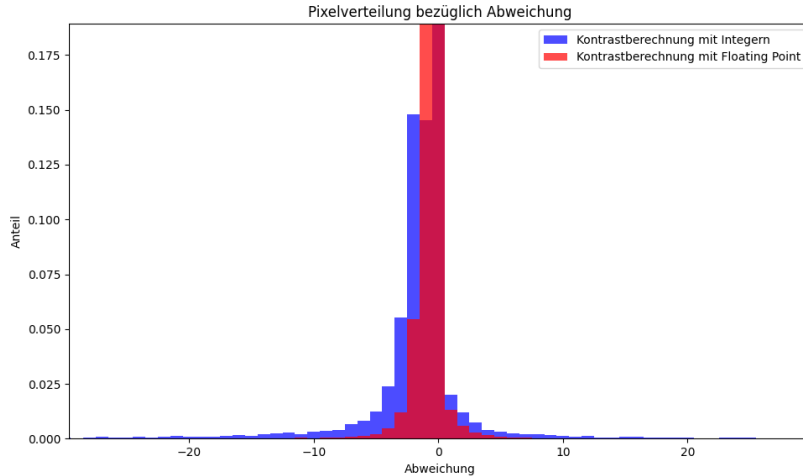
- $\mu - \frac{\sigma}{k} * \mu$  Vorberechnen → je nach Fall eine Addition oder Subtraktion mit Integer

# Festkommarechnung

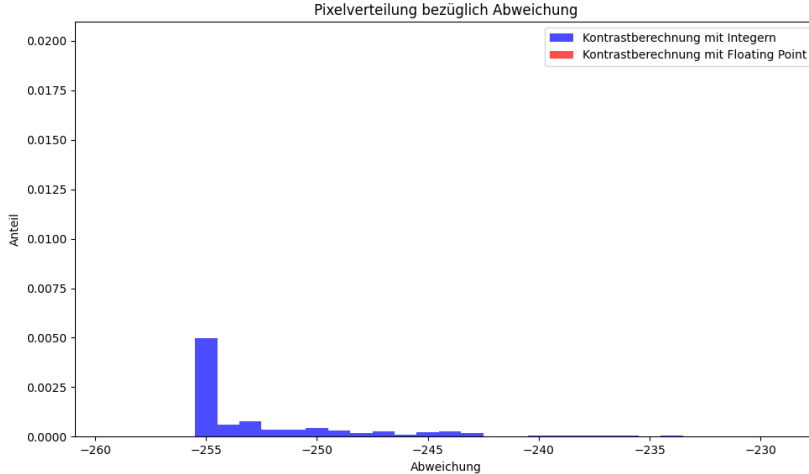
- Keine Vektorisierung → Festkommazahlen können in `uint64_t` gespeichert werden, da `Double` 52 Mantissenbits hat → 12.52 Festkommazahlen
- 12.52-Festkommazahlen → Multiplikation mit `uint8_t` Pixelwerten ohne Overflow möglich
- Shifts runden nicht → schnelle Festkommarundung per Bitmaske
  - ☐ Bestimmen der Vorkommastellen durch Shift
  - ☐ Isolieren der ersten Nachkommastelle
  - ☐ Shift dieses Bits an Position null
  - ☐ Addieren auf Vorkommastellen

- 1 Problemstellung
- 2 Lösungsansatz und Designentscheidungen
- 3 Optimierungen
- 4 Korrektheit und Genauigkeit**
- 5 Performanzanalyse



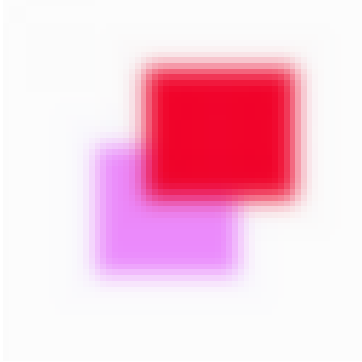


# Abweichung - Bereich bei 255





# Beispiel - Abweichungen an Übergängen



**(a)** Eingabe



**(b)** Basisimplementierung



**(c)** SSE mit Fließkommazahlen

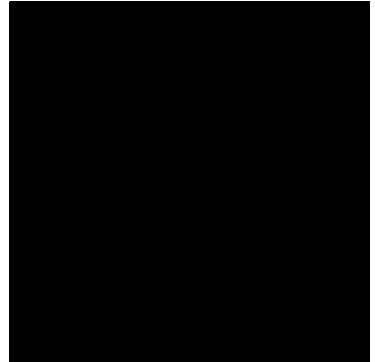
## Beispiel - kleines $\sigma$



**(a)** Eingabe



**(b)** Basisimplementierung



**(c)** AVX mit Fließkommazahlen

- 1 Problemstellung
- 2 Lösungsansatz und Designentscheidungen
- 3 Optimierungen
- 4 Korrektheit und Genauigkeit
- 5 Performanzanalyse**

